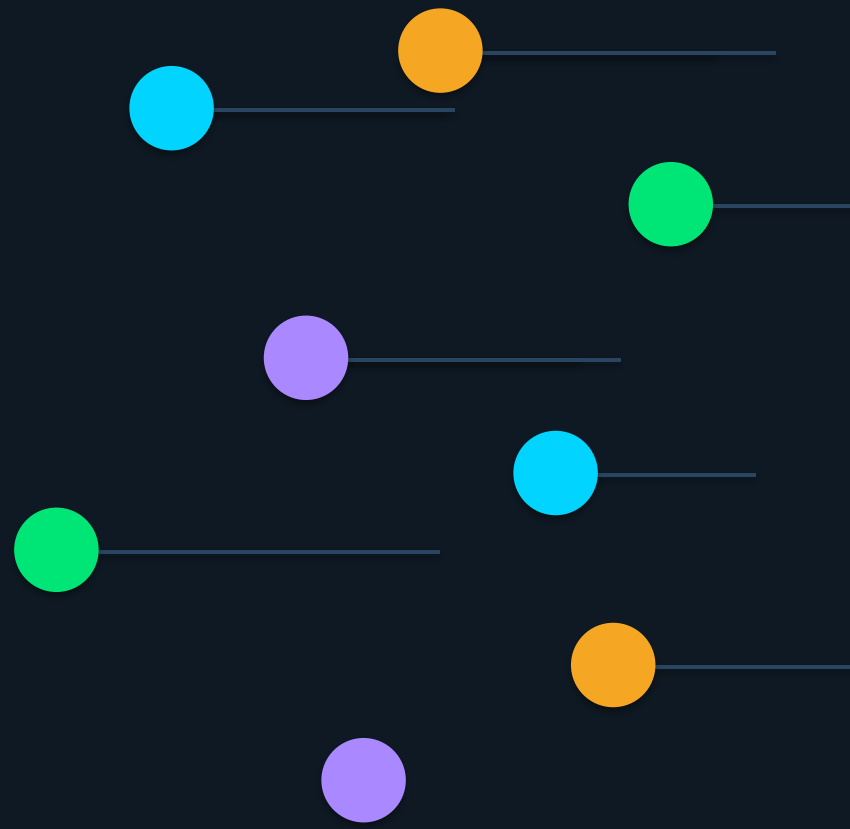
Fundamentos de Arquitectura LLM

Tema 3 · Consideraciones de costos

Tema 4 · Patrones y escalabilidad



Dos temas, una decisión: elegir bien antes de construir

TEMA 3

Consideraciones de Costos

- Iceberg de costos: visible vs. oculto
- Modelos de precios y referencia oct-2026
- Fórmula TCO: CAPEX + VP(OPEX) + Riesgo
- ROI ajustado y período de recuperación
- Framework multicriterio LATAM (7 criterios)
- Lab 05: evaluación de proveedores
- Lab 06: TCO y ROI a 24 meses

TEMA 4

Patrones y Escalabilidad

- Monolito vs. microservicios en LLMs
- 5 patrones esenciales: Gateway, Circuit Breaker…
- Contenedores multi-stage: -78% tamaño
- Escalado horizontal vs. vertical
- Kubernetes vs. Docker Swarm
- Lab 07: microservicios y contenedores
- Lab 08: autoscaling y orquestación

Al terminar estos temas usted podrá…

Calcular	el TCO a 24 meses para los tres escenarios de despliegue LLM principales
Evaluar	proveedores usando un framework multicriterio ponderado adaptado al contexto LATAM
Estimar	el ROI ajustado por riesgo y el período de recuperación de la inversión
Implementar	los patrones API Gateway, Circuit Breaker y Sidecar de forma conceptual
Decidir	cuándo usar escalado horizontal vs. vertical y K8s vs. Docker Swarm

CASO DE ESTUDIO DEL BLOQUE

TechCorp Latinoamérica

Empresa manufacturera con sedes en Bogotá, Lima y Ciudad de México. Evaluó LLMs en T1-T2 y ahora necesita responder: ¿Cuánto cuesta realmente y cómo lo desplegamos sin que se caiga?

PREGUNTA QUE GUIARÁ EL BLOQUE

¿Cuál escenario de despliegue tiene el menor TCO a 24 meses y la arquitectura más resiliente para 3.000 consultas/mes?

1 Mesa de ayuda de TI

3.000 consultas/mes
→ ¿API pública o on-premise?

2 Manuales técnicos

20.000 páginas indexadas
→ ¿RAG nube vs. local? TCO

3 Calidad

Reportes de no-conformidad
→ latencia, SLA y costos ocultos

4 Gobierno de datos

3 países, 3 leyes
→ cumplimiento, privacidad, lock-in

3

Consideraciones de Costos

Evaluación de proveedores · TCO/ROI · Análisis de riesgo para despliegues LLM en contexto LATAM

El iceberg de costos de un despliegue LLM

VISIBLE (lo que se cotiza)

- ✓ Precio por token (input / output)
- ✓ Suscripción mensual o PTU
- ✓ Hosting básico de la aplicación

OCULTO (el costo real)

- ▸ Ingeniería: prompts, guardrails, evaluación
- ▸ Cumplimiento regulatorio (LGPD, Ley 25.326, LFPDPPP)
- ▸ Latencia: conversión perdida por respuestas lentas
- ▸ Integración con sistemas legados
- ▸ Personal MLOps / LLMOps
- ▸ Costos de migración y lock-in

Regla práctica

2× Para presupuestar, duplica el costo de tokens estimado para cubrir costos de ingeniería y operación.

3–6m Antes de producción, asigna 3 a 6 meses de PoC para medir los costos ocultos reales.

20% Reserva un 20% del OPEX anual para mantenimiento de prompts, drift de modelos y evaluación de calidad.

5a El análisis de lock-in debe cubrir 5 años mínimo, no solo el primer año de contrato.

Fuente: Gartner AI Deployment Cost Framework, 2024

Modelos de precios en el mercado LLM

Modelo de precio	Ejemplo de cálculo	Ventaja	Desventaja
Pay-per-token (más común)	$0.15 / 1M tk input $0.60 / 1M tk output → 1M tk/día ≈ $22.5/mes	✓ Sin inversión inicial. Paga solo lo que usas.	✗ Costo impredecible con volumet creciente.
Provisioned Throughput (PTU)	Azure: $2/hora reservada → $1,440/mes fijo por capacidad comprometida	✓ Latencia garantizada, costo fijo y predecible.	✗ Pagas aunque no uses toda la capacidad.
Suscripción por asiento	$20-30/usuario/mes → Empresa 200 usuarios ≈ $4,000-6,000/mes	✓ Presupuesto fácil de aprobar y justificar.	✗ Escala mal con alto volumen automatizado.
CAPEX On-Premise (GPU + software)	A100 80GB: $15K–$30K + $3K-5K/mes OPEX (infra + personal)	✓ Control total. Cero costo por token.	✗ Alta inversión inicial. Requiere equipo MLOps.

Referencia de precios por modelo — oct 2026 (ilustrativo)

Modelo	Proveedor	Input $/1M tk	Output $/1M tk	Velocidad	Contexto	Perfil
GPT-4o mini	OpenAI	$0.15	$0.60	Alta	128K	Precio/rendimiento
Gemini 1.5 Flash	Google	$0.075	$0.30	Alta	1M	Más económico
Claude 3 Haiku	Anthropic	$0.25	$1.25	Alta	200K	Alta velocidad
Llama 3.1 70B	HuggingFace	$0.59	$0.79	Media	128K	Open source
GPT-4o	OpenAI	$2.50	$10.00	Media	128K	Calidad premium
Gemini 1.5 Pro	Google	$1.25	$5.00	Media	2M	Contexto largo
Claude 3.5 Sonnet	Anthropic	$3.00	$15.00	Media	200K	Razonamiento
Mixtral 8×22B	HuggingFace	$0.65	$0.65	Alta	64K	MoE eficiente

⚠ Datos ilustrativos — verificar en openai.com/pricing · ai.google.dev/pricing · anthropic.com/pricing · huggingface.co/pricing

Framework TCO — Total Cost of Ownership

CAPEX	OPEX	RIESGO
▸ Hardware / GPU	▸ Tokens / uso de API	▸ Prob. abandono del proveedor
▸ Licencias perpetuas	▸ Hosting e infraestructura	▸ Cambio regulatorio imprevisto
▸ Consultoría de setup	▸ Personal MLOps / LLMOps	▸ Costo de migración (lock-in)
▸ Adaptadores e integración	▸ Mantenimiento y actualizaciones	▸ Degradación del modelo
▸ Formación inicial	▸ Auditorías y cumplimiento	▸ Incidente de seguridad

TCO = CAPEX + Σ [OPEX(t) / (1 + r)^t] para t = 1 … N meses

- r = tasa de descuento mensual = (1 + tasa_anual)^(1/12) − 1
- N = horizonte de análisis (recomendado: 24 meses para LLMs)
- OPEX(t) incluye crecimiento de uso: OPEX(t) × (1 + g)^t donde g ≈ 3-8%/mes

ROI = (Beneficio Neto / TCO) × 100 **ROI ajustado = ROI × (1 − p_fallo) − p_fallo × (Impacto / TCO × 100)**

Comparativa TCO 24 meses — empresa LATAM 200 empleados, 1M tokens/día

	API Pública (Gemini Flash / GPT-4o mini)	Plataforma Gestionada (Azure OpenAI / AWS Bedrock)	On-Premise (GPU A100 + Ollama/vLLM)
CAPEX	$10,000	$28,000	$54,000
OPEX/mes (año 1)	$1,800	$3,200	$4,500
OPEX/mes (año 2, +5%/mes)	$2,200	$3,900	$4,500 (fijo)
TCO 24 meses	$52,000	$111,000	$162,000
Beneficio anual estimado	$70,000	$120,000	$145,000
ROI (24m)	~160%	~110%	~88%
Payback	**~6 meses**	**~9 meses**	**~18 meses**
Cumplimiento LATAM	Bajo	Medio-Alto	Alto
Recomendado para	PoC, startups	Banco, salud	Alto volumen ≥3 años

Los valores son ilustrativos. Tasas de descuento: 15% anual. Crecimiento de uso: 5% mensual.

ROI ajustado por riesgo — cuadrante de decisión

RIESGO BAJO · ROI ALTO

- Caso de uso bien definido
- Proveedor maduro
- Datos no sensibles
- Equipo técnico capacitado

✅ **PROCEDER**

RIESGO ALTO · ROI ALTO

- Procesamiento de datos sensibles
- Integración con core bancario
- Dependencia de un solo proveedor
- Regulación en evolución

⚠️ **MITIGAR RIESGO**

◀ ROI BAJO — ROI ALTO ▶

RIESGO BAJO · ROI BAJO

- Caso de uso genérico
- Bajo impacto en el negocio
- No diferenciador competitivo
- Solución existente satisfactoria

🔄 **REVALUAR**

RIESGO ALTO · ROI BAJO

- Alta complejidad técnica
- Regulación muy estricta
- ROI incierto o negativo
- Sin sponsor ejecutivo

🚫 **NO PROCEDER**

7 criterios de evaluación de proveedores — contexto LATAM

01	**Costo por token**	Input + output por cada 1M tokens; proyectar a 12 meses con crecimiento esperado.
02	**Cumplimiento regulatorio**	LGPD (BR), Ley 25.326 (AR), LFPDPPP (MX), sector financiero/salud.
03	**Latencia P95**	Percentil 95 de tiempo de respuesta. Meta: <2s P95 para chat, <10s para batch.
04	**Ecosistema y madurez**	SDKs, documentación, comunidad, integrations. Tiempo en mercado.
05	**Privacidad de datos**	¿Retienen los datos para entrenamiento? Residencia de datos en LATAM.
06	**Soporte técnico**	SLA, canales en español, tiempo de resolución. Crítico en producción.
07	**Soporte en español/portugués**	Calidad del modelo en idiomas locales. Probar con casos de uso reales.

Lab 05: evaluar los 7 criterios con datos ilustrativos de 7 proveedores → python labs/sesion2/05_evaluacion_proveedores.py

Síntesis Tema 3 — ¿Cuándo elegir cada escenario?

🚀 PoC / Startup	🏛️ Producción Regulada	⚡ Alto Volumen ≥3 años
API Pública (Gemini Flash, GPT-4o mini)	**Plataforma Gestionada (Azure OpenAI, AWS Bedrock)**	**On-Premise (A100 + Ollama / vLLM)**
✓ Inversión inicial mínima (<$10K)	✓ Cumplimiento LGPD / Ley 25.326 / LFPDPPP certificado	✓ Costo por token cercano a cero después del payback
✓ ROI positivo en 4–6 meses	✓ SLA empresarial con soporte 24/7	✓ Control total: privacidad, auditoría, personalización
✓ Iteración rápida sin infraestructura	✓ Residencia de datos en región LATAM disponible	✓ Requiere equipo MLOps dedicado (2–3 personas)
✓ Ideal cuando el modelo de negocio aún no está validado	✓ ROI positivo en 8–12 meses	✓ ROI positivo a partir del mes 18–24
Riesgo: lock-in y escalabilidad de costos al crecer.	*Riesgo: costo crece con el volumen; evaluar PTU al mes 6.*	*Riesgo: inversión inicial alta; tecnología puede quedar obsoleta.*

Lab 06: calcula automáticamente el TCO/ROI para los 3 escenarios → python labs/sesion2/06_tco_y_roi.py

Regla de oro: nunca elijas proveedor por precio/token solo. Calcula el TCO a 24 meses e incluye costos regulatorios y de migración.

4

Patrones y Escalabilidad

Microservicios · Contenedores · HPA / KEDA
Kubernetes vs Docker Swarm · Integración asíncrona

Monolito vs microservicios en aplicaciones LLM

MONOLITO

- Frontend
- Lógica de negocio + LLM calls
- Auth + Rate limit
- Base de datos
- Caché + Embeddings

✓ Un solo despliegue
✓ Más simple de empezar

✗ Escala todo o nada
✗ Falla del LLM tumba todo
✗ Actualizaciones con downtime

MICROSERVICIOS

- 🌐 API Gateway
- 🔐 Auth Service
- 💬 LLM Service (Ollama / API)
- 📊 Embeddings
- 💾 Cache Service
- 📋 Queue Worker

✓ Escala por servicio individualmente
✓ Fallo aislado (Circuit Breaker)
✓ Despliegue independiente

✗ Mayor complejidad operacional
✗ Latencia de red entre servicios

Recomendación: Comenzar con monolito modular. Extraer microservicios solo cuando un componente necesita escalar independientemente.

5 patrones arquitectónicos esenciales para LLMs en producción

01 API Gateway

Punto único de entrada. Centraliza autenticación, rate-limiting, enrutamiento A/B y logging de todas las llamadas al LLM.

02 Circuit Breaker

Detecta fallos repetidos y 'abre el circuito' (CERRADO → ABIERTO → SEMIABIERTO). Evita cascada de errores cuando el proveedor falla.

03 Sidecar

Contenedor auxiliar en el mismo pod que agrega logging, métricas y mTLS sin modificar el servicio principal. 'Extiende sin tocar'.

04 Caché Semántica

Embeddings de queries + similitud coseno. Si existe respuesta con similitud >0.92, retornarla. Ahorra 40-70% de tokens en FAQs y soporte.

05 Worker Asíncrono

API → Cola (Kafka/SQS) → Pool de workers → LLM. Para batch, documentos largos y procesos que toleran latencia.

Lab 07: simulación de Gateway + Circuit Breaker → python labs/sesion2/07_microservicios_y_contenedores.py

Contenedores multi-stage — reducción de tamaño y superficie de ataque

❌ **Dockerfile monolítico (~1.5 GB)**

```
FROM python:3.12

RUN apt-get install -y gcc g++ git curl

WORKDIR /app

COPY requirements.txt .

RUN pip install -r requirements.txt

COPY . .

CMD ["uvicorn", "app.main:app"]
```

Incluye: gcc, g++, git, curl, cabeceras de compilación

✅ **Multi-stage (~335 MB, −78%)**

```
# — Stage 1: builder —

FROM python:3.12 AS builder

COPY requirements.txt .

RUN pip install --prefix=/install -r requirements.txt

# — Stage 2: runtime —

FROM python:3.12-slim

COPY --from=builder /install /usr/local

COPY . /app

RUN useradd -m appuser && chown appuser /app

CMD ["uvicorn", "app.main:app"]
```

Tamaño: 1.5 GB → 335 MB (−78%) | Vulnerabilidades: ↓ drásticamente | Tiempo de pull en CD: 5× más rápido | Usuario no-root: ✓ seguridad mejorada

Principio: el stage de BUILD instala dependencias; el stage RUNTIME solo copia los binarios compilados.

Escalado horizontal vs vertical — ¿cuándo usar cada uno?

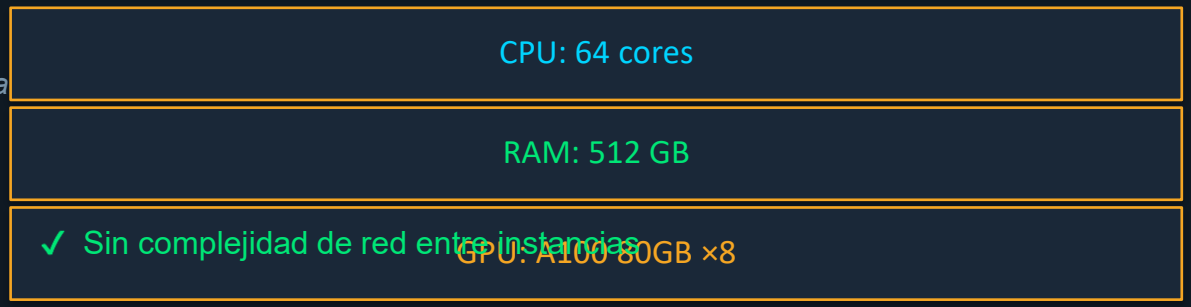
↔ ESCALADO HORIZONTAL (scale-out)



- ✓ Capacidad teóricamente ilimitada
- ✓ Alta disponibilidad inherente (fallo de 1 pod no afecta al sistema)
- ✓ Rolling updates sin downtime
- ✓ Costo escala linealmente con la carga
- ✗ El estado compartido es complejo (DB, caché, sesiones)
- ✗ Requiere load balancer y orquestador

Ideal para: APIs stateless (FastAPI backend, frontend)

↕ ESCALADO VERTICAL (scale-up)



- ✓ Sin complejidad de red entre instancias
- ✓ Necesario para modelos grandes (>70B params)
- ✓ Tensor parallelism nativo en una máquina
- ✗ Limitado por el hardware más grande disponible
- ✗ SPOF: si falla la máquina, cae todo el servicio
- ✗ Costo escala superlinealmente

Ideal para: LLMs grandes (Llama 70B+), modelos con estado

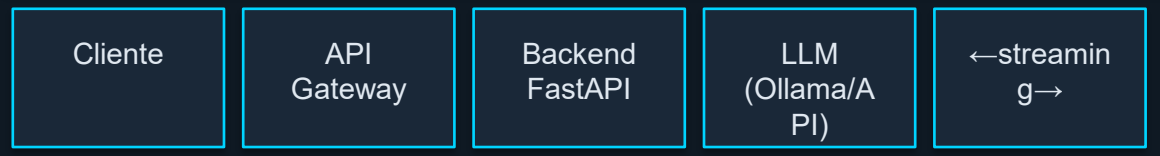
En la práctica: escalado horizontal para el backend API; escalado vertical para el servidor LLM con GPU.

Kubernetes vs Docker Swarm — guía de selección

Criterio	Docker Swarm	Kubernetes
Curva de aprendizaje	Baja — 1-2 días para un equipo con Docker	Alta — 2-4 semanas mínimo
Setup inicial	1 comando: docker swarm init	kubeadm / EKS / GKE / AKS
Auto-scaling	Manual (docker service scale)	HPA + KEDA (scale-to-zero)
GPU scheduling	Básico (--generic-resources)	Avanzado (nvidia plugin, MIG)
Multi-cluster	No soportado	Sí (Federation, Submariner)
Observabilidad nativa	Básica (docker stats)	Prometheus + Grafana + Jaeger
Rolling updates	Sí, configurable	Sí + canary + blue/green
Overhead operacional	Muy bajo	Alto (etcd, API server, etc.)
Ecosistema de herramientas	Limitado	Enorme (Helm, Istio, ArgoCD…)
Recomendado para	MVP, <5 servicios, equipo pequeño	Producción, >10 servicios, GPU

Integración síncrona vs asíncrona con sistemas existentes

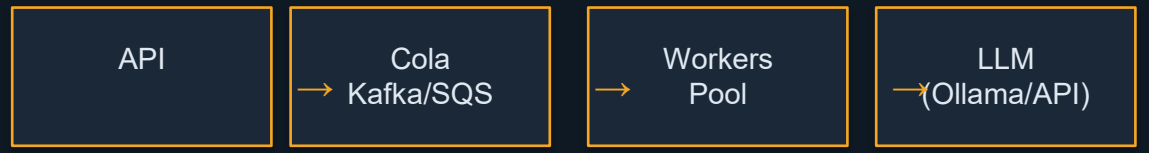
SÍNCRONO — REST / SSE streaming



- ✓ Chat en tiempo real (respuesta en <3s)
- ✓ Búsqueda semántica interactiva
- ✓ Generación de texto corto (emails, resúmenes)
- ✓ API pública con clientes esperando respuesta
- ✗ Timeout del cliente si LLM tarda >30s
- ✗ No apto para documentos largos (>50 páginas)

Tech: FastAPI SSE · WebSockets · gRPC streaming

ASÍNCRONO — Cola de mensajes



Resultado → Webhook / DB / Notificación push

- ✓ Análisis de documentos largos (contratos, reportes)
- ✓ Procesamiento masivo (1000+ documentos/hora)
- ✓ Generación de reportes programados
- ✓ Pipelines ETL con enriquecimiento LLM

Brokers disponibles en LATAM:

- Kafka, RabbitMQ, AWS SQS
- Redis Streams, Google Pub/Sub

Lab 08: simula cola de mensajes Kafka/SQS + autoscaler → python labs/sesion2/08_escalabilidad_y_orquestacion.py

Arquitectura de referencia — LLM en producción para LATAM

Capa 1 — Clientes	Web (React)	Mobile (React Native)	API Partners	Sistema Legado
Capa 2 — API Gateway + CDN	Traefik / Kong	Auth (JWT/OAuth)	Rate Limit	Cache HTTP
Capa 3 — Microservicios Backend	FastAPI (×N réplicas)	Circuit Breaker	Caché Semántica	Observabilidad
Capa 4 — LLM Inference	Ollama/vLLM (GPU)	API Pública (OpenAI/Gemini)	Load Balancer	Fallback
Capa 5 — Datos y Cola	PostgreSQL / Vector DB	Kafka / SQS	Redis (Caché)	Object Storage

Orquestación: Kubernetes (HPA + KEDA scale-to-zero) · Observabilidad: Prometheus + Grafana · CI/CD: GitHub Actions / Azure DevOps

Síntesis T4 — 10 conceptos que todo arquitecto LLM debe dominar

API Gateway

Punto único de entrada para auth, rate-limit y routing.

Circuit Breaker

Evita cascada de fallos cuando el LLM falla.

Sidecar

Extiende servicios sin modificarlos (logs, mTLS, métricas).

Caché Semántica

Embeddings + similitud para reusar respuestas. Ahorra 40-70% tokens.

Multi-stage Build

Imágenes 78% más pequeñas. Sin herramientas de build en runtime.

Horizontal Scaling

N réplicas para APIs stateless. HPA automático en K8s.

Vertical Scaling

Más GPU/RAM para el modelo LLM. Necesario para >70B params.

HPA + KEDA

Auto-scaling por CPU/mem y por eventos (Kafka lag, SQS depth).

Docker Swarm

Orquestador simple para MVP. 1 comando para arrancar.

Kubernetes

Producción. HPA, KEDA, GPU scheduling, multi-cluster.